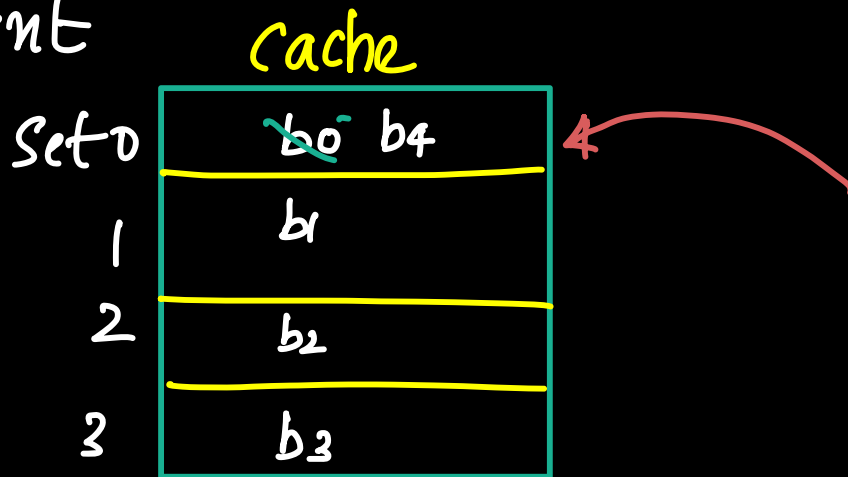


Cache Replacement.

(Note)

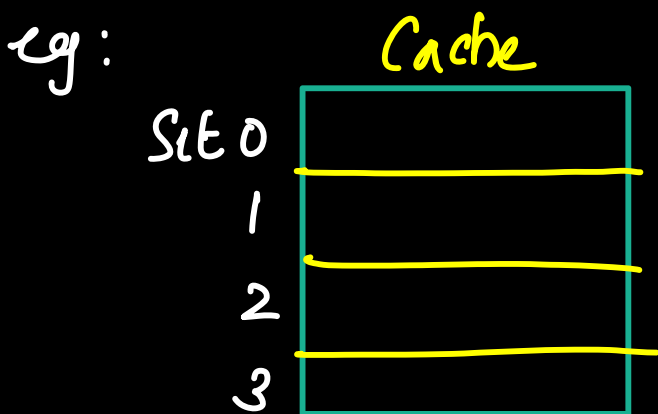
- When a new data needs to be loaded into the cache and the cache set is already full, some existing data must be evicted (removed) from the cache to make space for the new data. The process of deciding which block to remove from the cache is called cache replacement.



- The need for cache mapping depends on the cache mapping technique used.

→ Cache replacement in direct mapping.

- in direct mapping each block in main memory is mapped to a particular set in the cache.
- there is one possible location for each memory block in the cache.



Assume block requests
b₀, b₁, b₂, b₃, b₄ (Blocks of MM)

b₀ is mapped to set 0

b₀ → set 0
b₁ → set 1
b₂ → set 2
b₃ → set 3

Cache	
Set 0	b ₀
1	b ₁
2	b ₂
3	b ₃

When block b₄ is requested, it should be mapped to Set '0'. But set '0' already full (contains b₀). So b₀ is automatically replaced by the b₄. (2)

Cache	
Set 0	b₀ b ₄
1	b ₁
2	b ₂
3	b ₃

— In direct mapping no replacement is required. The mapped block is always replaced.

Cache replacement in associative mapping

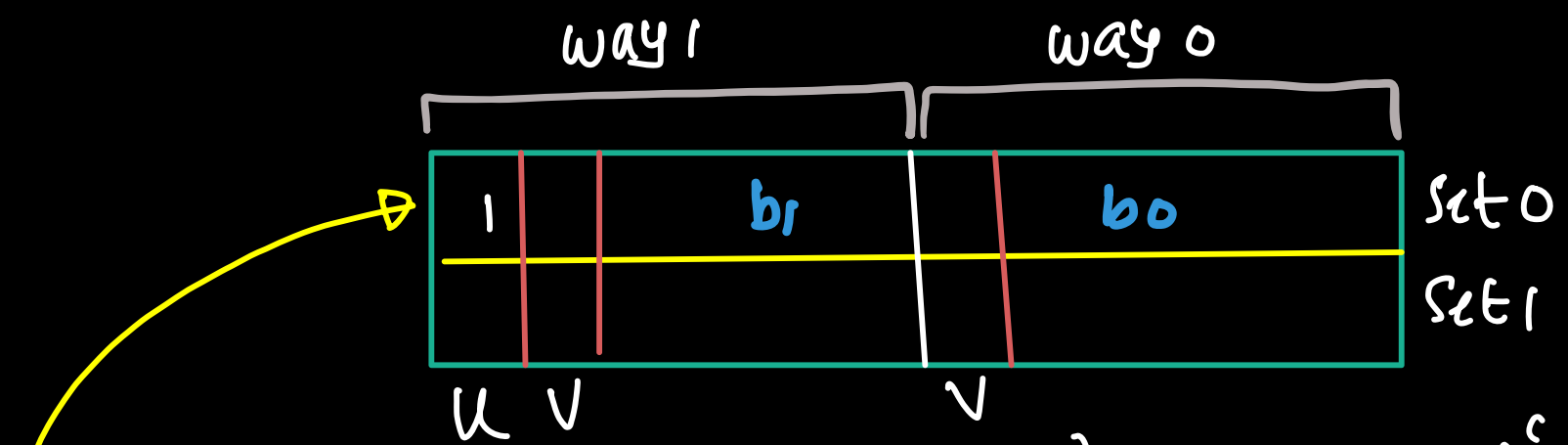
— In set associative mapping, cache is divided into sets, each set contains more than one block. A memory block can be placed any block within the set.

— When a set is full and new data must be loaded, the cache must decide which block within the set should be evicted. Therefore a replacement policy is required.

— The principle of temporal locality suggests that data accessed recently is likely to be accessed again soon. Therefore best choice for replacement is the block that was ^{not} recently used, as it is least likely to be used again in the near future.

— Hence most associative caches use the Least recently used (LRU) replacement policy.

- Assume main memory blocks b_0, b_1, b_2 are mapped to N same cache set (set 0)
- Assume block requests $b_0, b_1, b_0, b_2 \dots$
 - first block b_0 is mapped to set 0 (way 0)
 - 2nd block b_1 is mapped to set 0 (way 1)
 - Now set 0 is full



- Next request is block ' b_0 '. It is in set '0' (way 0) so block ' b_1 ' in way 1 becomes least recently used. so use bit become 1 ($u=1$) for indicating data in way is LRU.

- Next Block request is b_2 , but set '0' is full. use bit of that set is checked
- use bit $u=1$, i.e. data in way 1 in that set is LRU and replace that block b_1 with b_2

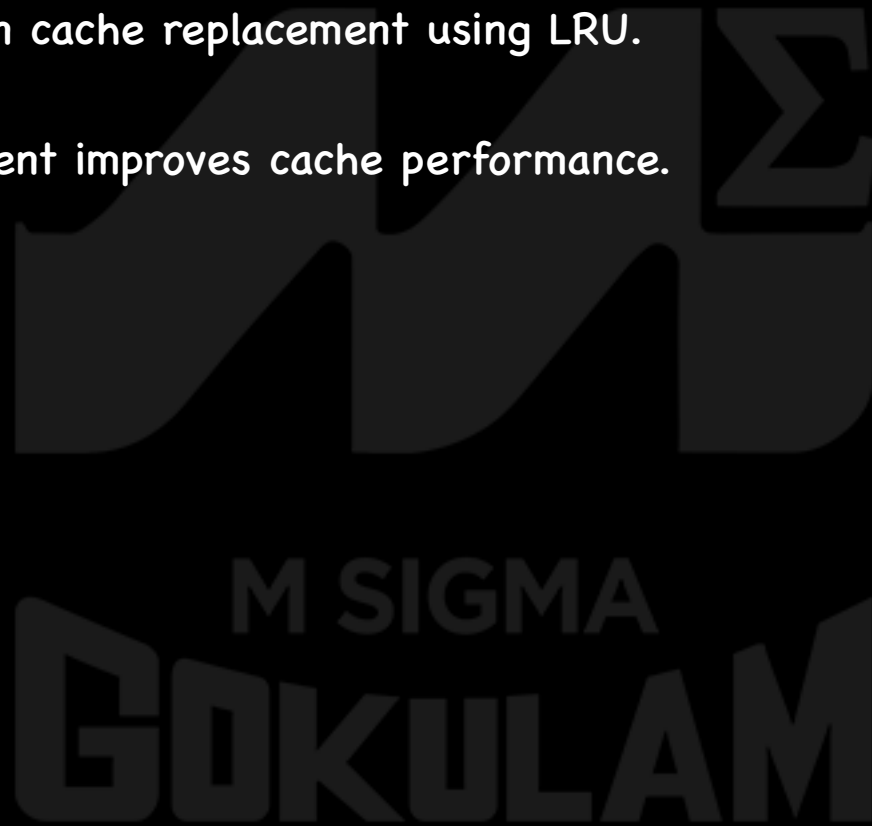
- If one block is accessed, the use bit is updated to mark other block as LRU.
- when replacement is needed, the block marked as LRU is replaced.

Advantages of LRU

- ① Reduces the misses
- ② Improve the cache performance.

Sample questions- cache replacement

- Explain cache replacement policies in detail with examples.
- Describe LRU replacement policy and its implementation in a 2-way set associative cache.
- Explain cache replacement in:
 - Direct mapped cache
 - Set associative cache
 - Fully associative cache
- Discuss LRU replacement policy
- Explain the advantages and disadvantages of LRU.
- With a neat diagram, explain cache replacement using LRU.
- Explain how cache replacement improves cache performance.



Multi Level cache:

Note

- the performance of a memory system can be evaluated using average memory access time (AMAT).
- To improve the performance we need to reduce AMAT
- If Large cache are using, then they can store more data and hence reduce the miss rate. But access time is Large.
- If small cache are using, then the access time is very small, but the storage capacity is limited, leading to a higher miss rate.

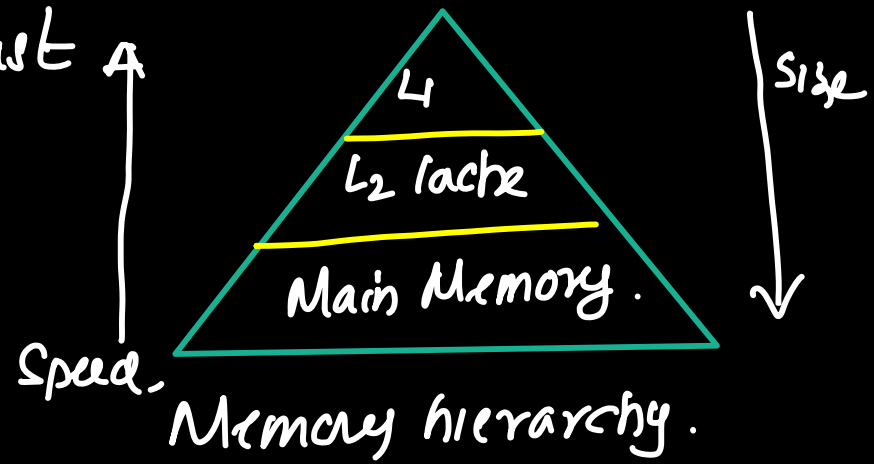
	Capacity	AMAT	Miss rate
Large cache	Large	Large	Low
Small cache	Small	small	High.

- To balance speed and capacity, modern computer uses multiple level of cache memory. This approach helps to reduce the AMAT.
- Need of multi level cache
 - Small cache — very fast but limited capacity
 - Large cache — Large capacity but slower.
- To combine advantages of both, system use multi level caching
 - small and fast cache is placed very close to CPU.
- Organisation of multi level cache.
 - A typical two level cache system consists of the following components
 - ① Level 1 (L_1) cache
 - ② Level 2 (L_2) "

- Level 1 (L_1) cache
 - Located closest to the processor
 - very small and very fast

- Level 2 (L_2) cache

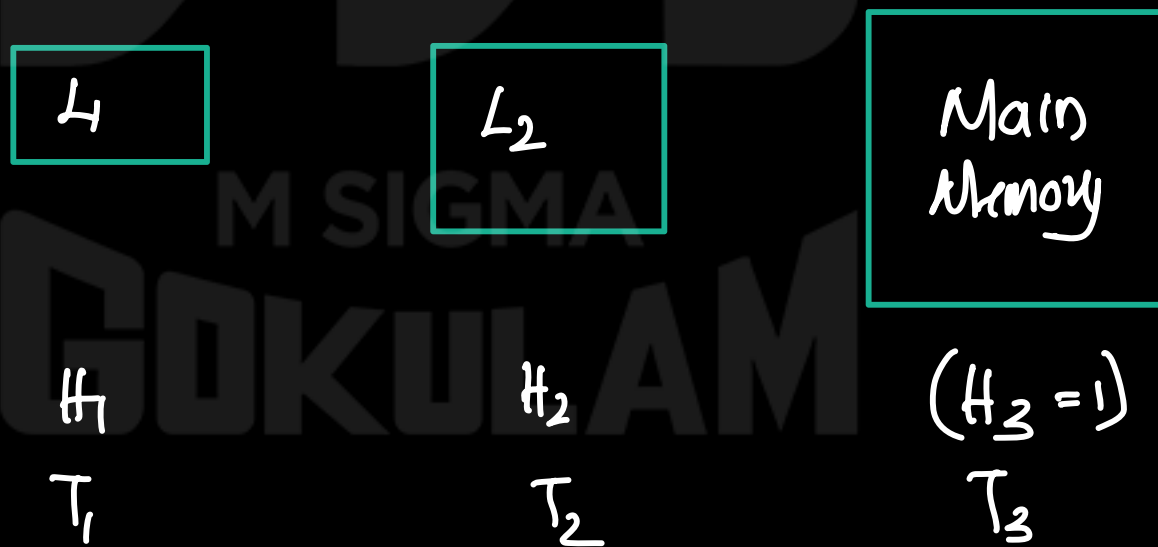
- Larger than L_1
- slower than L_1 .



Working of multilevel cache

- ① processor first check the L_1 cache
- ② if the required data is found, then cache hit occurs
- ③ if the L_1 cache misses, processor checks the L_2 cache.
- ④ if the L_2 cache misses, data fetched from the memory.

Performance Analysis of two level cache
→ evaluated using AMAT



$$AMAT = H_1 T_1 + (1 - H_1) H_2 (T_1 + T_2) + (1 - H_1)(1 - H_2) H_3 (T_1 + T_2 + T_3)$$

Given:

L1 cache access time = 1 cycle

L2 cache access time = 10 cycles

Main memory access time = 100 cycles

L1 miss rate = 5%

L2 miss rate = 20% (of L1 misses)

$$T_1 = 1 \text{ cycle}$$

$$T_2 = 10 \text{ cycle}$$

$$T_3 = 100 \text{ cycle}$$

$$MR_1 = \frac{5}{100} = 0.05$$

$$H_1 = 1 - 0.05 = \underline{\underline{0.95}}$$

$$MR_2 = \frac{20}{100} = \underline{\underline{0.2}}$$

$$H_2 = 1 - 0.2 = \underline{\underline{0.8}}$$

$$AMAT = H_1 T_1 + (1 - H_1) H_2 (T_1 + T_2) + (1 - H_1) (1 - H_2) H_3 (T_1 + T_2 + T_3)$$

$$= 0.95 \times 1 + 0.05 \times 0.8 (1 + 10) + 0.05 \times 0.2 \times 1 (1 + 10 + 100)$$

$$= 0.95 + 0.44 + 0.01 (111)$$

$$= 0.95 + 0.44 + 1.11$$

$$= 2.5 \text{ cycles} \rightarrow T \text{ is given as cycles}$$

→ Advantage of Multi level cache

- ① Reduce AMAT
- ② increase the overall performance of memory sys.

Sample questions

Explain multi-level cache architecture with a neat diagram.

Discuss the performance of a two-level cache system using AMAT.

Explain the organisation, working, and advantages of multi-level cache.

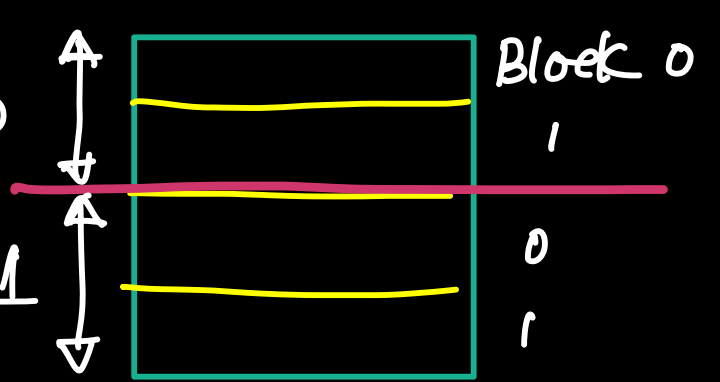
Compare single-level cache and multi-level cache systems.

Reducing cache Misses

Note

— Cache performance is affected by Miss Rate

$$\text{Miss Rate} = MR = \frac{\text{no. of Misses}}{\text{Total No. of access.}}$$

—  → cache miss can be reduced by

- ① Cache size (Eg: 16 byte)
- ② Block size (Eg: 4 Byte)
- ③ associativity (Eg: no. of Blocks in set = 2)

— types of Misses

- ① Compulsory Misses
- ② Capacity Misses
- ③ Conflict Misses

① Compulsory Misses

→ Let say the block request is b_0, b_1



initially block has not mapped.

— initially no block are not mapped to cache.
→ so when block request b_0 it is not present in the cache. thus a miss occur. this miss is called compulsory miss

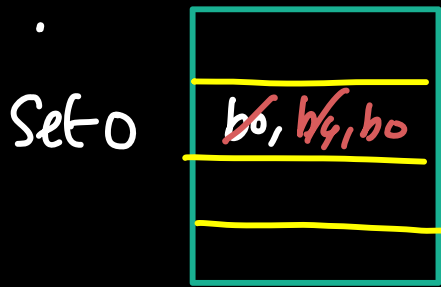
— this miss occur when memory accessed for the first time.

— increasing block size reduces the compulsory misses

② Conflict Miss.

— This occurs when multiple memory blocks map to the same cache set, evict block that are still needed

eg: Block request b_0, b_4, b_0, b_4



→ First request b_0 — cache hit
→ 2nd Request b_4 → Replace b_0 (Miss)

→

b_0	b_4
1	2

3rd Request b_0 → Miss (already b_0 present but replaced)

~~b_0~~ , ~~b_4~~ , b_0

— Conflict Miss can be reduced by increasing the cache capacity.
(cache capacity ↑, no. of set ↑)

③ Capacity Misses

- This miss occurs when the cache size is too small to hold all the data blocks that are needed
- Compulsory Miss can be reduced by increasing cache size

Effect of cache parameter on Cache Miss (Note)

Cache parameter.

- ① Cache capacity
- ② Block size
- ③ Associativity

① Cache capacity

- Increase in cache capacity reduces capacity miss and reduce conflict miss
- Does not affect compulsory misses.

② Block size

- increasing block size reduces the compulsory misses.
- increase block size increases conflict misses

③ Associativity

- increasing associativity means, increasing the no. of blocks in a set
- For small cache increasing the associativity, decreases the conflict misses

Summary

	Compulsory	Conflict	Capacity
Cache size ↑	—	↓ ②	↓ ①
block size ↑	↓ ②	↑	—
associativity ↑	—	↓	—



Sample questions

Explain the different types of cache misses and methods to reduce them.

Discuss how cache capacity, block size, and associativity affect cache miss rate.

Explain cache miss behavior

